

# 张翔 — 课程答辩代码详细说明

---

## 目录

---

1. 我的职责概述
  2. 数据结构基础
  3. 销售模块 (sale.c) 逐行讲解
  4. 进货模块 (purchase.c) 逐行讲解
  5. 前端 API 层 (api.js)
  6. 前端操作页面 (views-actions.jsx)
  7. 前端表格管理 (views-tables.jsx)
  8. 答辩常见问题预案
- 

## 1. 我的职责概述

---

根据小组分工，我（张翔）负责：

| 模块   | 文件                                                        | 说明                 |
|------|-----------------------------------------------------------|--------------------|
| 销售管理 | <code>backend/sale.c</code> / <code>sale.h</code>         | 销售记录的增删改查 + 库存联动扣减 |
| 进货管理 | <code>backend/purchase.c</code> / <code>purchase.h</code> | 进货记录的增删改查 + 库存联动增加 |
| 前端交互 | <code>frontend/js/api.js</code>                           | 前后端通信封装            |
| 前端页面 | <code>frontend/js/views-actions.jsx</code>                | 新建销售页、录入进货页        |
| 前端表格 | <code>frontend/js/views-tables.jsx</code>                 | 交易记录表、进货记录表的展示与操作  |

**一句话概括我的工作：** 我负责了系统中"钱"和"货"流动的核心业务逻辑——卖出去（销售出库）、买进来（进货入库），以及这两套动作与库存的实时联动，再加上对应的前端交互界面。

---

## 2. 数据结构基础

---

在讲我的代码之前，需要理解系统定义的数据结构（定义在 `data.h` 中）：

## 2.1 三个核心结构体

```
// 商品结构体
typedef struct {
    char id[64];           // 商品编号 P001
    char name[64];         // 商品名称
    char category[64];     // 类别
    char manufacturer[64]; // 厂家
    char model[64];        // 型号
    int stock;             // 库存数量 ← 我操作的字段
    double price;          // 单价
} Product;

// 销售记录结构体
typedef struct {
    char id[64];           // 交易编号 S001
    char product_id[64];   // 关联商品编号 ← 用于找商品
    char name[64];         // 商品名称 (冗余存储)
    char category[64];     // 类别 (冗余存储)
    char date[64];         // 交易日期 YYYY-MM-DD
    int quantity;          // 销售数量
    double price;          // 实际售价
    double total;          // 总金额 = quantity × price
} Sale;

// 进货记录结构体 (与 Sale 结构完全对称)
typedef struct {
    char id[64];           // 进货编号 B001
    char product_id[64];   // 关联商品编号
    char name[64];         // 商品名称 (冗余存储)
    char category[64];     // 类别 (冗余存储)
    char date[64];         // 进货日期
    int quantity;          // 进货数量
    double price;          // 进货单价
    double total;          // 总金额
} Purchase;
```

## 2.2 全局数组 (我操作的数据)

```
extern Product  products[MAX_PRODUCTS]; // 商品数组
extern int      product_count;           // 现有商品数

extern Sale     sales[MAX_SALES];        // 销售记录数组
extern int      sale_count;              // 现有销售记录数

extern Purchase purchases[MAX_PURCHASES]; // 进货记录数组
extern int      purchase_count;          // 现有进货记录数
```

**答辩时可以说：** 系统没有用数据库，而是用结构体数组 + 管道符分隔的文本文件来存数据。三个全局数组就是系统的“内存数据库”，我写的销售和进货模块就是在这些数组上做增删改查，同时联动修改商品的 stock 字段。

## 2.3 我调用的关键函数

```
// 数据层提供的函数
int find_product_by_id(const char *id); // 按编号查商品，返回数组下标，找不到返回 -1
void next_sale_id(char *buf);           // 生成下一个销售编号 S001...
void next_purchase_id(char *buf);       // 生成下一个进货编号 B001...
void save_sales(void);                  // 保存销售记录到文件
void save_purchases(void);             // 保存进货记录到文件
void save_products(void);              // 保存商品数据到文件
```

## 3. 销售模块 (sale.c) 逐行讲解

### 3.1 文件结构和依赖

```
#include <stdio.h> // fopen, fgets, fprintf, sprintf
#include <stdlib.h> // qsort (快速排序)
#include <string.h> // strcmp, strncpy, strtok, strlen
#include "data.h"   // 数据结构定义和全局数组
#include "utils.h"  // JSON构建器、URL解码
#include "product.h" // 商品查找函数
#include "sale.h"   // 自己的头文件
```

**答辩要点：**为什么#include？因为要用 `qsort()` 标准库的快速排序函数来实现交易记录的日期排序。

### 3.2 辅助函数

#### 3.2.1 sale\_to\_json() — 序列化函数

```
static void sale_to_json(const Sale *s, JsonBuilder *jb) {
    json_begin_obj(jb);           // 开始一个 JSON 对象 {
    json_add_str(jb, "id", s->id); // "id": "S001"
    json_add_str(jb, "product_id", s->product_id);
    json_add_str(jb, "name", s->name);
    json_add_str(jb, "category", s->category);
    json_add_str(jb, "date", s->date);
    json_add_int(jb, "quantity", s->quantity);
    json_add_dbl(jb, "price", s->price);
    json_add_dbl(jb, "total", s->total);
    json_end_obj(jb);             // 结束对象 }
}
```

**答辩要点：**这个函数把内存中的 C 结构体变成 JSON 字符串，是前后端通信的关键。`static` 关键字表示这个函数只在 sale.c 内部使用，不对外暴露——这是 C 语言的封装思想。

### 3.2.2 qsort 比较函数

```
// 升序：按日期从早到晚排列
static int cmp_sale_date_asc(const void *a, const void *b) {
    return strcmp(((Sale *)a)->date, ((Sale *)b)->date);
}

// 降序：按日期从晚到早排列
static int cmp_sale_date_desc(const void *a, const void *b) {
    return strcmp(((Sale *)b)->date, ((Sale *)a)->date);
}
```

答辩要点：

- `qsort()` 需要比较函数的签名是 `int cmp(const void *a, const void *b)` —— 两个 `void *` 参数，返回整数。
- 返回值规则：`<0` 表示 a 排前面，`>0` 表示 b 排前面，`=0` 表示顺序不变。
- 为什么直接用 `strcmp` 比较日期？因为我们的日期格式是 `YYYY-MM-DD`，从左到右是“年→月→日”，字典序 = 时间序。例如 `"2026-05-01" < "2026-05-27"` 在字典序里天然成立。
- 降序就是把 a 和 b 的位置换一下——`strcmp(b->date, a->date)`。

这个技巧展示了对字符串比较和数据格式设计的理解。

## 3.3 handle\_get\_sales() — 多条件查询 + 排序

这是销售模块最“长”的函数，但它做的事情很清晰：

URL 查询参数 → 解析参数 → 遍历数组筛选 → qsort 排序 → 构建 JSON 响应

步骤1：解析 URL 查询参数

```
void handle_get_sales(const char *query, char *response_body) {
    // 声明变量存放解析结果，全部初始化为空
    char category[64] = {0};    // 类别筛选
    char date_from[64] = {0};   // 起始日期
    char date_to[64] = {0};     // 结束日期
    char product_id[64] = {0};  // 按商品编号筛选
    char sort[16] = "desc";     // 默认降序
    int has_category = 0, has_from = 0, has_to = 0, has_pid = 0;
    // 用标志位记录“用户是否传了这个参数”，用于后面判断
}
```

然后用 `strtok()` 按 `&` 切分参数字符串：

```

char *tok = strtok(qcopy, "&"); // 第一次调用 strtok
while (tok) {
    url_decode(tok); // 把 %20 之类的转义还原
    if (strncmp(tok, "category=", 9) == 0) {
        strncpy(category, tok + 9, 63); // tok+9 跳过 "category="
        has_category = 1;
    }
    // ... 类似处理其他参数
    tok = strtok(NULL, "&"); // 后续调用传入 NULL
}

```

答辩要点：

- `strtok()` 是 C 标准库的字符串分割函数。第一次调用传字符串，后续传 `NULL` 继续切割同一字符串。它会修改原字符串（插入 `\0`）。
- `strncmp(tok, "category=", 9)` 比较前 9 个字符是否匹配。用 `strncmp` 而非 `strcmp` 是因为 `strncmp` 只比较前 n 个字符，更高效也防止意外。
- `tok + 9` 是指针偏移：跳过 "category=" 这 9 个字符，让指针直接指向后面的值部分。

步骤2：多条件"与"逻辑筛选

```

Sale filtered[MAX_SALES]; // 临时数组存放筛选结果
int fcount = 0;
for (int i = 0; i < sale_count; i++) {
    int match = 1; // 默认匹配
    if (has_category && strcmp(sales[i].category, category) != 0) match = 0;
    if (has_from && strcmp(sales[i].date, date_from) < 0) match = 0;
    if (has_to && strcmp(sales[i].date, date_to) > 0) match = 0;
    if (has_pid && strcmp(sales[i].product_id, product_id) != 0) match = 0;
    if (match) filtered[fcount++] = sales[i];
}

```

答辩要点：

- 所有条件是"与"的关系——必须同时满足才会加入结果。
- `has_category` 等标志位的作用：如果用户没有传这个参数，`has_category` 就是 0，对应的条件就直接跳过（短路求值），不影响筛选。
- 日期比较用 `strcmp`：因为日期格式是 `YYYY-MM-DD`，`strcmp` 逐字符比较 ASCII 码，恰好等价于日期先后比较。

步骤3：排序

```

if (strcmp(sort, "asc") == 0)
    qsort(filtered, fcount, sizeof(Sale), cmp_sale_date_asc);
else
    qsort(filtered, fcount, sizeof(Sale), cmp_sale_date_desc);

```

答辩要点：

- `qsort()` 是 C 标准库中的快速排序，定义在 `<stdlib.h>` 中。
- 四个参数：① 数组首地址 ② 元素个数 ③ 每个元素大小（`sizeof(Sale)`）④ 比较函数。

- 为什么用 `sizeof(Sale)` ? `qsort` 需要知道每个元素多大, 才能在交换元素时移动正确的字节数。

#### 步骤4 : 构建 JSON 响应

```
char buf[131072]; // 128KB 缓冲区
JsonBuilder jb;
json_init(&jb, buf, sizeof(buf));
json_begin_obj(&jb);
json_add_int(&jb, "total", fcount); // "total": 42
json_begin_array(&jb, "data"); // "data": [
for (int i = 0; i < fcount; i++) {
    // 每个销售记录单独序列化
    char item_buf[512];
    JsonBuilder item_jb;
    json_init(&item_jb, item_buf, sizeof(item_buf));
    sale_to_json(&filtered[i], &item_jb); // 调用辅助函数
    if (!jb.first) json_append_raw(&jb, ","); // 不是第一个就加逗号
    json_append_raw(&jb, item_buf);
    jb.first = 0;
}
json_end_array(&jb); // ]
json_end_obj(&jb); // }
```

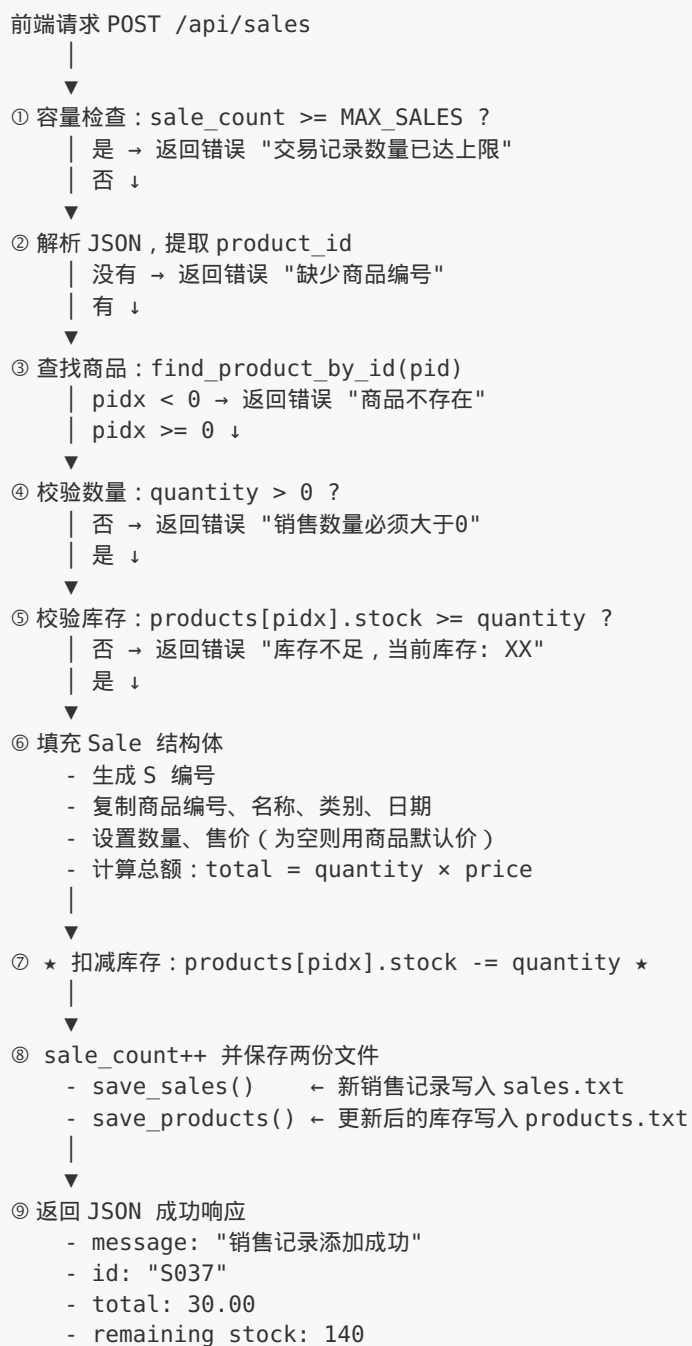
#### 答辩要点 :

- 这里展示了本系统自己实现的 `JsonBuilder` (工具层代码) 的使用方法。
- `jb.first` 是一个标志位, 用来判断是不是 JSON 的第一个字段/元素——如果不是, 前面要加逗号分隔。
- 最终返回的 JSON 格式 : `{"total": 42, "data": [{...}, {...}, ...]}`

### 3.4 `handle_create_sale()` — 创建销售 (核心业务逻辑)

这是整个系统中**最核心的函数**, 体现了"销售出库 + 库存联动"的完整流程。

## 流程图



关键代码逐行解释



```

void handle_create_sale(const char *body, char *response_body) {
    // 步骤1: 容量检查
    if (sale_count >= MAX_SALES) {
        sprintf(response_body, "{\"error\":\"交易记录数量已达上限\"}");
        return;
    }

    // 步骤2: 解析商品编号
    char pid[64];
    if (!json_get_str(body, "product_id", pid, 64)) {
        sprintf(response_body, "{\"error\":\"缺少商品编号\"}");
        return;
    }

    // 步骤3: 查找商品
    int pidx = find_product_by_id(pid);
    if (pidx < 0) {
        sprintf(response_body, "{\"error\":\"商品不存在\"}");
        return;
    }

    // 步骤4: 校验数量
    int qty = json_get_int(body, "quantity");
    if (qty <= 0) {
        sprintf(response_body, "{\"error\":\"销售数量必须大于0\"}");
        return;
    }

    // 步骤5: ★ 关键——校验库存是否充足 ★
    if (products[pidx].stock < qty) {
        sprintf(response_body, "{\"error\":\"库存不足, 当前库存: %d\"}",
products[pidx].stock);
        return;
    }

    // 步骤6: 填充销售记录
    Sale *s = &sales[sale_count]; // 指向数组末尾的空位
    next_sale_id(s->id);           // 生成 S001 格式的编号
    strncpy(s->product_id, pid, 63);
    // ★ 冗余复制名称和类别 ★
    strncpy(s->name, products[pidx].name, 63);
    strncpy(s->category, products[pidx].category, 63);
    json_get_str(body, "date", s->date, 64);
    s->quantity = qty;
    s->price = json_get_dbl(body, "price");
    if (s->price <= 0) s->price = products[pidx].price; // 默认用商品售价
    s->total = s->quantity * s->price;

    // 步骤7: ★ 扣减库存 ★
    products[pidx].stock -= qty;

    // 步骤8: 计数+1, 保存两份文件
    sale_count++;
    save_sales(); // 保存 sales.txt
    save_products(); // 保存 products.txt (库存已更新)

    // 步骤9: 返回成功响应
    JsonBuilder jb;
    json_init(&jb, response_body, 8192);
    json_begin_obj(&jb);
    json_add_str(&jb, "message", "销售记录添加成功");
    json_add_str(&jb, "id", s->id);
}

```

```

    json_add_dbl(&jb, "total", s->total);
    json_add_int(&jb, "remaining_stock", products[pidx].stock);
    json_end_obj(&jb);
}

```

答辩核心要点：

1. **为什么要先做所有校验，最后才修改数据？** 这是防御性编程——如果先扣库存再发现数量不合法，库存已经错了。所有校验通过后才执行实际修改。
2. **为什么要同时保存两个文件？** 销售记录增加了，商品库存也变化了。如果只保存 sales.txt 不保存 products.txt，程序重启后库存就回到旧值，造成数据不一致。
3. **为什么在 Sale 结构体里冗余存储 name 和 category？** 这是典型的"用空间换时间"。如果只存 product\_id，每次显示销售列表时都要去商品数组里查对应的名称和类别，效率低。冗余存储后，查销售记录就是一张"自包含"的表，不需要跨表关联。
4. **为什么售价可以为空？** 如果前端没传售价或传了 0，就使用商品的默认售价。这是为了让"正常售价销售"更便捷——用户只需选商品和数量。但同时也保留了修改售价的能力（议价场景）。

### 3.5 handle\_update\_sale() — 修改销售（库存回滚）

修改销售记录时，数量变了，库存也要跟着调整。我采用的策略是"先恢复旧扣减，再应用新扣减"：

```

旧数量 = 10件，库存 = 100，新数量 = 7件

① stock += 10 → 库存变成 110 （撤销旧销售）
② 检查 110 >= 7 ? → 是，通过
③ stock -= 7 → 库存变成 103 （应用新销售）
④ 保存

```

关键代码：

```

int qty = json_get_int(body, "quantity");
if (qty > 0) {
    int pidx = find_product_by_id(s->product_id);
    if (pidx >= 0) {
        // ① 先恢复旧库存
        products[pidx].stock += s->quantity;
        // ② 检查新数量是否可行
        if (products[pidx].stock < qty) {
            // ★ 库存不足：回滚恢复操作 ★
            products[pidx].stock -= s->quantity;
            sprintf(response_body, "{\"error\":\"库存不足\"}");
            return;
        }
        // ③ 扣减新数量
        products[pidx].stock -= qty;
        save_products();
    }
    s->quantity = qty;
}
}

```

#### 答辩要点：

- 这个方法比"直接计算差值 `stock += (old - new)`"更容易理解和调试，异常分支也更清晰。
- 如果新数量超了，必须把第一步恢复的库存再扣回去，否则库存就虚高了。这是"回滚"思想。
- 如果用户只改了日期没改数量，`qty > 0` 不满足，库存不受影响。

### 3.6 handle\_delete\_sale() — 删除销售（退货/库存恢复）

```
void handle_delete_sale(const char *id, char *response_body) {
    // ① 找到要删除的销售记录
    int idx = -1;
    for (int i = 0; i < sale_count; i++) {
        if (strcmp(sales[i].id, id) == 0) { idx = i; break; }
    }
    if (idx < 0) {
        sprintf(response_body, "{\"error\":\"未找到交易记录\"}");
        return;
    }

    // ② ★ 恢复库存：把扣减的量加回去 ★
    int pidx = find_product_by_id(sales[idx].product_id);
    if (pidx >= 0) {
        products[pidx].stock += sales[idx].quantity;
        save_products();
    }

    // ③ 数组前移删除（不是真正释放内存，而是覆盖）
    for (int i = idx; i < sale_count - 1; i++) {
        sales[i] = sales[i + 1]; // 后面的元素往前挪
    }
    sale_count--; // 计数减1
    save_sales(); // 保存

    sprintf(response_body, "{\"message\":\"交易记录删除成功\"}");
}
```

#### 答辩要点：

- 删除销售 = 退货场景。退货后库存应该恢复，所以 `stock += quantity`。
- 删除的方式：数组前移覆盖。这不是“释放内存”，而是把后面的元素一个个往前搬，然后把计数减 1。这是 C 语言数组管理的经典做法。
- 注意操作的顺序：**先恢复库存，再删除记录**。如果反过来，删了记录之后再想恢复库存，已经找不到 `quantity` 了。

## 4. 进货模块 (purchase.c) 逐行讲解

### 4.1 与销售模块的对称关系

这是答辩中最容易被问到的问题。我的设计和回答：

| 操作 | 销售 (sale.c)     | 进货 (purchase.c) |
|----|-----------------|-----------------|
| 创建 | 库存 -= 数量 (出库)   | 库存 += 数量 (入库)   |
| 修改 | 先恢复旧扣减 → 应用新扣减  | 先撤销旧增量 → 应用新增量  |
| 删除 | 库存 += 数量 (退货恢复) | 库存 -= 数量 (退购扣回) |

核心思想：销售和进货形成“镜像对称”——销售是出库操作，进货是入库操作，删除各自是反操作。

## 4.2 handle\_create\_purchase() — 创建进货

流程与 `handle_create_sale()` 几乎一样，只有两个关键差异：

**差异1：不需要检查库存是否充足**

```
// sale.c 需要检查：
if (products[pidx].stock < qty) {
    return error "库存不足";
}
// purchase.c 不需要！进货是增加库存，不存在“不够”的问题
```

**差异2：库存操作方向相反**

```
// sale.c
products[pidx].stock -= qty;    // 减库存

// purchase.c
products[pidx].stock += qty;    // 加库存
```

除这两个差异外，编号生成（B 编号 vs S 编号）、冗余复制、保存两份文件的逻辑完全一致。

## 4.3 handle\_delete\_purchase() — 删除进货（多了库存下限保护）

```
// * 扣回库存 *
int pidx = find_product_by_id(purchases[idx].product_id);
if (pidx >= 0) {
    products[pidx].stock -= purchases[idx].quantity;
    // * 底线保护：库存不能是负数 *
    if (products[pidx].stock < 0) products[pidx].stock = 0;
    save_products();
}
```

**答辩要点：**

- 为什么删除进货需要库存下限保护，但删除销售不需要？因为删除销售是恢复库存（加回来），只可能越来越多，不可能变负数。但删除进货是扣回库存——如果进货之后已经卖了很多，当前库存可能比进货量少，扣回就会变成负数。比如进了 100 件笔，卖了 90 件剩 10 件，这时候删除进货记录， $10 - 100 = -90$ ，不设下限就炸了。
- 这是一个**防御性编程**的体现：宁可库存为 0（保守），也不能出现负数（业务上无意义）。

## 5. 前端 API 层 (api.js)

### 5.1 整体架构

```
(function () {  
  const BASE_URL = "http://localhost:8080/api";  
  // ... 工具函数  
  // ... 数据规范化  
  // ... API 方法  
  window.StationeryApi = Api; // 挂载到全局  
})();
```

答辩要点：

- 用 **IIFE**（立即执行函数表达式）包裹，避免污染全局作用域。所有内部变量如 `BASE_URL`、`request()` 都是私有的。
- 最后只暴露一个 `window.StationeryApi` 对象，这是 JavaScript 的模块化思想。

### 5.2 核心：request() 函数

```
async function request(path, options = {}) {  
  const { params, body, headers, timeoutMs = 8000, ...rest } = options;  
  const controller = new AbortController();  
  const timer = setTimeout(() => controller.abort(), timeoutMs); // 超时机制  
  let response;  
  try {  
    response = await fetch(buildUrl(path, params), {  
      ...rest,  
      signal: controller.signal,  
      headers: { "Content-Type": "application/json", ...(headers || {}) },  
      body: body === undefined  
        ? undefined  
        : (typeof body === "string" ? body : JSON.stringify(body)),  
    });  
  } catch (err) {  
    if (err?.name === "AbortError") throw new Error("请求后端超时，请确认服务已启动");  
    throw new Error("请求后端失败：" + (err?.message || "网络连接异常"));  
  } finally {  
    clearTimeout(timer); // 无论成功失败都要清除定时器  
  }  
  
  // 统一错误处理  
  const raw = await response.text();  
  let data = JSON.parse(raw);  
  if (!response.ok || data?.error) {  
    throw new Error(data?.error || data?.message || `请求失败 (${response.status})`  
  );  
  }  
  return data;  
}
```

### 答辩要点：

- **AbortController + setTimeout** 实现了请求超时控制。如果后端没响应，8 秒后自动取消请求，不会让用户一直等。
- **finally 中 clearTimeout** 很重要。不管成功还是失败，定时器都要清除，否则会内存泄漏——定时器握着 controller 的引用不放。
- 错误处理分三层：网络层（catch 捕获 fetch 失败）、协议层（response.ok 检查 HTTP 状态码）、业务层（data.error 检查后端返回的业务错误）。

## 5.3 数据规范化

```
// 前端内部用驼峰命名 (productId, qty, maker)
// 后端用下划线命名 (product_id, quantity, manufacturer)
// normalizeRecord/denormalizeRecord 做双向转换

function normalizeRecord(row = {}) {
  return {
    id: row.id,
    productId: row.product_id ?? row.productId ?? "", // 兼容两种格式
    name: row.name,
    category: row.category,
    date: row.date,
    qty: toNumber(row.quantity ?? row.qty),
    price: toNumber(row.price),
    total: row.total === undefined ? qty * price : toNumber(row.total),
  };
}
```

### 答辩要点：

- 前后端字段命名风格不同（前端驼峰 vs 后端下划线），规范化函数充当“翻译层”。
- **??** 是空值合并运算符，只有左边是 **null** 或 **undefined** 才取右边。比 **||** 更精确，因为 **0** 和空字符串 **""** 是合法的值。

## 5.4 销售/进货 API 方法

```
createSale: (record) => request("/sales", {
  method: "POST",
  body: denormalizeRecord(record),
}),
updateSale: (id, record) => request(`/sales/${encodeURIComponent(id)}`, {
  method: "PUT",
  body: denormalizeRecord(record, { includeProduct: false }), // 修改不传product_id
}),
deleteSale: (id) => request(`/sales/${encodeURIComponent(id)}`, {
  method: "DELETE",
}),
```

### 答辩要点：

- 修改销售时 **includeProduct: false** ——修改销售记录不应该改变它关联的商品，所以请求体里不包含 **product\_id** 字段。

- `encodeURIComponent(id)` 防止编号中有特殊字符导致 URL 解析错误。

## 6. 前端操作页面 (views-actions.jsx)

这个文件包含两个 React 组件：`SellView`（新建销售）和 `RestockView`（录入进货）。

### 6.1 SellView — 新建销售页

状态管理

```
const [productId, setProductId] = useState(""); // 选中的商品
const [qty, setQty] = useState(1); // 销售数量
const [price, setPrice] = useState(0); // 售价
const [date, setDate] = useState(today(0)); // 日期，默认今天
const [pending, setPending] = useState(false); // 提交中状态
```

售价联动

```
useEffect(() => {
  if (product) setPrice(product.price); // 换商品时自动填充默认售价
}, [productId]);
```

**答辩要点：** 用户选择商品后，售价自动填充为商品默认售价，方便"标准售价销售"。同时售价框可手动修改，支持"议价"场景。

提交逻辑

```
const submit = async () => {
  // ① 前端校验（和后端校验形成双重保护）
  if (!product) { toast("请选择商品", "err"); return; }
  if (qty <= 0) { toast("数量必须大于 0", "err"); return; }
  if (qty > product.stock) { toast("库存不足，仅剩 " + product.stock, "err");
return; }

  setPending(true);
  try {
    await StationeryApi.createSale({ productId: product.id, date, qty, price });
    await Promise.all([refreshProducts(), refreshTransactions()]); // 并行刷新
    toast(`已售出 ${qty} 件 · ${product.name} · ${fmtMoney(subtotal)}`);
    // 清空表单
    setProductId(""); setQty(1); setPrice(0);
  } catch (err) {
    toast(err?.message || "销售提交失败", "err");
  } finally {
    setPending(false);
  }
};
```



答辩要点：

- `Promise.all([refreshProducts(), refreshTransactions()])` 并行刷新两个数据源，比串行快。
- 提交成功后清空表单，用户可以连续录入多笔销售。
- 前端也做了库存校验，这是**双重保险**——前端校验提升体验（立即反馈），后端校验保障数据安全（防止绕过页面直接调 API）。

## 6.2 RestockView — 录入进货页

该页面有**两种模式**：

**模式1：已有商品补货** — 选择已有商品，输入数量和进价，直接创建进货记录。

**模式2：新品入库** — 先用 `createProduct` 创建新商品，再用 `createPurchase` 创建进货记录。两步操作是一个"事务"：

```
const created = await StationeryApi.createProduct({ ...newItem, stock: 0 });
const productId = created?.id;
await StationeryApi.createPurchase({ productId, date, qty, price });
```

**答辩要点：** "新品入库"模式是业务需求的延伸——文具店进了新品牌、新商品，既要加商品信息，又要加进货记录，还要联动库存。先创建商品（stock 设为 0），再创建进货（stock += quantity），最终库存=进货量。

---

## 7. 前端表格管理 (views-tables.jsx)

这个文件中的 `TransactionsView` 和 `PurchasesView` 是我写的核心。两个组件的结构非常相似。

### 7.1 TransactionsView — 交易记录管理

关键功能

1. **多条件筛选**：关键字搜索 + 类别筛选
2. **排序**：支持按日期升序/降序，也支持按金额、数量等字段排序
3. **修改**：弹出 Modal 表单，调用 `StationeryApi.updateSale()`
4. **删除**：弹出确认框，调用 `StationeryApi.deleteSale()`

## 修改交易的关键代码

```
const save = async () => {
  if (editing.qty <= 0) { toast("数量必须大于 0", "err"); return; }
  setPending("save");
  try {
    await StationeryApi.updateSale(editing.id, editing);
    // ★ 修改后同时刷新交易列表和商品列表 ★
    await Promise.all([refreshTransactions(), refreshProducts()]);
    toast("已更新交易 · " + editing.id);
    setEditing(null);
  } catch (err) {
    toast(err?.message || "更新交易失败", "err");
  }
};
```

**答辩要点：** 为什么修改交易后要同时刷新交易列表和商品列表？因为修改数量会联动改变库存，商品列表显示的库存数字变了，所以需要两个都刷新。

## 删除交易的关键代码

```
const doRemove = async () => {
  await StationeryApi.deleteSale(confirm.id);
  await Promise.all([refreshTransactions(), refreshProducts()]);
  toast("已删除交易 · " + confirm.id);
};
```

删除确认框的提示信息明确告诉用户：“后端会自动恢复对应库存”——这就是我在 C 后端写的 `stock += quantity`。

## 7.2 TransactionsView 和 PurchasesView 的本质区别

两个组件结构几乎相同，只是在以下细节有差异：

- API 调用不同：`createSale` vs `createPurchase`、`updateSale` vs `updatePurchase`
- 删除提示文字不同：交易删除说“恢复库存”，进货删除说“扣回库存”
- 统计指标不同：交易看“营收/revenue”，进货看“成本/cost”
- 进货表格没有“按日期排序”的快捷按钮

---

## 8. 答辩常见问题预案

**Q1：**为什么 **Sale** 结构体里要重复存 **name** 和 **category**，而不是只存 **product\_id**？

**回答：** 这是“反范式设计”——用冗余存储换查询效率。如果只存 `product_id`，每次展示销售列表时，都要拿 `product_id` 去商品数组里查对应的名称和类别，N 条记录就要查 N 次。冗余存储后，Sales 表是一张“自包含”的表，不需要跨表关联。代价是商品改名后，历史销售记录里的名称不会自动更新，但这个场景在本系统中极少发生。

**Q2：**如果有一个销售创建成功后，保存 **sales.txt** 成功但保存 **products.txt** 时断电了，会发生什么？

**回答：** 这是文本文件方案的固有限制——不提供数据库级别的事务保证。重启后会出现"销售记录已存在但库存未扣减"的不一致状态。在答辩中坦诚承认这一点，同时说明这是课程规模下的合理取舍。如果要改进，可以引入 SQLite + WAL 模式，利用数据库的事务机制保证原子性。

**Q3：**为什么删除销售时用数组前移而不是链表？

**回答：** C 语言结构体数组相比链表的优势：① 内存连续，CPU 缓存友好，遍历更快；② 不需要维护指针，代码更简单；③ 对于课程设计中最多 500 条记录的数据量，数组前移（最多搬动 500 个结构体）的性能开销可以忽略不计。如果数据量很大（百万级别），才需要考虑链表或标记删除。

**Q4：**为什么前端和后端都做了库存校验？

**回答：** 这叫"纵深防御"（Defense in Depth）。前端校验是用户体验层的，让用户在选择商品时就能看到错误提示，不用等到提交后才报错。后端校验是安全保障层的，因为 HTTP 请求可以绕过浏览器直接发送，不能信任前端传来的数据。两层校验目的不同，都是必要的。

**Q5：**修改销售记录时为什么要"先恢复旧扣减，再应用新扣减"，而不是直接算差值？

**回答：** "恢复-检查-应用"的三段式做法虽然比直接算差值多一步操作，但语义更清晰、异常分支更容易处理。直接算差值是 `stock += (old_qty - new_qty)`，但如果新数量非常大（比如用户误输入 99999），`old_qty - new_qty` 是一个很大的负数，stock 加上这个负数可能就变成负数了，而中间没有任何检查点。我的三段式做法在第二步有明确的库存检查，不会放过这种异常。

**Q6：**进货删除时为什么设 **stock** 下限为 0？

**回答：** 删除进货意味着"这笔进货不算了"，对应的库存要扣回去。但如果进货之后已经卖了很多（比如进了 100 件，已经卖了 90 件剩 10 件），扣回 100 件会让库存变成 -90。库存为负数在业务上没有意义——你不能有"负 90 件"库存。设下限为 0 是一个保守的数据完整性保护策略。

**Q7：****api.js** 里的 **normalizeRecord** 和 **denormalizeRecord** 是干什么的？

**回答：** 这是前后端数据格式的"翻译层"。后端 C 代码用下划线命名（`product_id`、`quantity`），前端 JavaScript 习惯用驼峰命名（`productId`、`qty`）。**normalize** 把后端格式转成前端格式，**denormalize** 反过来。这样做的好处是前后端各自用舒适的命名风格，通过翻译层解耦。

**Q8：**前端 **abort** 超时机制的实现原理是什么？

**回答：** 利用 `AbortController` API。创建一个 controller，把它的 signal 传给 fetch；同时用 `setTimeout` 在 8 秒后调用 `controller.abort()`。一旦 abort 触发，fetch 会抛出 `AbortError`。关键在于 `finally` 里的 `clearTimeout(timer)`——无论请求成功还是失败，都要清除定时器，否则定时器会一直挂在那里造成资源泄漏。

**Q9：**销售和进货的代码为什么要分开写，而不是用一个通用函数？

**回答：** 虽然两者流程相似，但有本质差异——销售要检查库存是否充足，进货不需要；销售删除时恢复库存，进货删除时要设下限保护。如果强行合并，代码里会充满 `if (is_sale)` 和 `if (is_purchase)` 分支，反而降低可读性。保持两个独立模块，代码虽然长一些，但语义清晰，维护和理解都更容易。

**Q10：**你负责的代码中用到了哪些 C 语言知识点？

**回答：** 结构体与 `typedef`（定义 Product/Sale/Purchase 数据模型）、结构体数组（内存数据存储）、指针操作（`Sale *s = &sales[sale_count]`）、字符串处理（`strcmp`、`strncpy`、`strtok`、`strncmp`）、标准库 `qsort`（快速排序 + 自定义比较函数）、文件 I/O 概念（通过调用 data 层接口）、静态函数 `static`（封装内部函数）、错误处理（层层校验 + 提前返回）、以及数组删除技巧（元素前移覆盖）。前端方面用了 `Promise/async-await`、`React Hooks`（`useState`、`useEffect`、`useMemo`）、`fetch` API、`AbortController`、`IIFE` 模块化等 JavaScript 知识点。

---

## 附录：答辩话术建议

---

开场（介绍自己负责的部分）

"我负责的是系统的销售管理模块、进货管理模块和前端交互界面。具体来说，我用 C 语言实现了销售和进货记录的增删改查，以及与商品库存的实时联动——卖东西自动扣库存，进货自动加库存，删除记录时恢复/扣回库存。前端方面，我写了新建销售、录入进货两个操作页面，以及交易记录和进货记录的表格管理页面，还有前后端通信的 API 封装层。"

核心亮点（主动展示）

"我代码中有几个值得关注的设计点：第一，销售和进货形成镜像对称的库存联动机制，销售出库扣减、进货入库增加；第二，修改记录时采用'先恢复后应用'的回滚式算法，不是简单算差值，而是完整的校验-恢复-扣减流程；第三，所有校验都放在数据修改之前，

确保任何一步校验失败都不会留下脏数据；第四，前端通过 *AbortController* 实现了请求超时机制，提升用户体验。"